

# ESCUELA POLITÉCNICA NACIONAL

## Laboratorio Práctico

### React Hooks y Axios desde cero

*Gestor de publicaciones con React + Vite + JavaScript*

|                   |                                                       |
|-------------------|-------------------------------------------------------|
| Asignatura        | Aplicaciones Web                                      |
| Tema              | React Hooks, Axios, servicios y custom hooks          |
| Modalidad         | Laboratorio guiado paso a paso                        |
| Duración sugerida | 2 a 3 horas                                           |
| Herramientas      | Node.js, npm, Visual Studio Code, Vite, React y Axios |

## 1. Tema del laboratorio

Construcción de una mini aplicación llamada Gestor de publicaciones. La aplicación permitirá consultar publicaciones desde una API pública, mostrarlas en pantalla, crear publicaciones simuladas, eliminar publicaciones de la interfaz y organizar el código usando componentes, servicios y custom hooks.

## 2. Objetivo general

Desarrollar una aplicación frontend con React que consuma una API REST mediante Axios, aplicando hooks fundamentales y una estructura de proyecto organizada.

## 3. Resultados de aprendizaje

- Crear un proyecto React moderno con Vite.
- Instalar y configurar Axios para realizar peticiones HTTP.
- Aplicar useState para manejar formularios, listas, carga y errores.
- Aplicar useEffect para ejecutar la carga inicial de datos.
- Separar la lógica de acceso a datos en servicios.
- Construir un custom hook para reutilizar lógica de negocio.
- Implementar renderizado condicional para carga, errores y listas vacías.
- Crear, listar y eliminar publicaciones en una interfaz React.

## 4. Conceptos que se aplican

| Concepto                | Aplicación en el laboratorio                          |
|-------------------------|-------------------------------------------------------|
| useState                | Manejar el formulario, publicaciones, carga y errores |
| useEffect               | Cargar datos al iniciar el componente                 |
| Axios                   | Consumir una API REST                                 |
| Servicios               | Separar la lógica de comunicación con la API          |
| Custom Hook             | Reutilizar la lógica de publicaciones                 |
| Componentes             | Organizar la interfaz visual                          |
| Renderizado condicional | Mostrar carga, error o datos                          |
| Eventos                 | Enviar formulario y eliminar publicaciones            |

## 5. API que se utilizará

Se usará la API pública JSONPlaceholder, específicamente el recurso /posts:

```
https://jsonplaceholder.typicode.com/posts
```

**Nota:** JSONPlaceholder simula operaciones de creación y eliminación. La API responde como si la operación se hubiera realizado, pero no persiste los datos en el servidor.

## 6. Resultado esperado

Al finalizar, la aplicación debe mostrar un formulario para crear publicaciones, una lista de publicaciones obtenidas desde la API, mensajes de carga/error y un botón para eliminar publicaciones de la lista local.

Gestor de publicaciones con Hooks y Axios

Título: \_\_\_\_\_  
Contenido: \_\_\_\_\_  
[Crear publicación]

Publicaciones registradas

- Título de publicación  
Contenido de publicación  
[Eliminar]

## 7. Paso 1: Crear el proyecto React

Abra la terminal en la carpeta donde desea crear el proyecto y ejecute:

```
npm create vite@latest laboratorio-posts-react
```

Seleccione las siguientes opciones:

React  
JavaScript

```
P3E004-K@39682082D20P3 MINGW64 ~/AplicacionesWeb
$ cd REACT-INTEGRATION/

P3E004-K@39682082D20P3 MINGW64 ~/AplicacionesWeb/REACT-INTEGRATION
$ npm create vite@latest laboratorio-posts-react

> npx
> create-vite laboratorio-posts-react

|
| ◊ Select a framework:
|   React
|
| ◊ Select a variant:
|   JavaScript
|
| ◊ Install with npm and start now?
|   Yes
|
| ◊ Scaffolding project in C:\Users\P3E004-K\AplicacionesWeb\REACT-INTEGRATION\laboratorio-posts-react...
|
| ◊ Installing dependencies with npm...
|
| added 135 packages, and audited 136 packages in 21s
|
| 31 packages are looking for funding
|   run `npm fund` for details
|
| found 0 vulnerabilities
|
| ◊ Starting dev server...
|
| > laboratorio-posts-react@0.0.0 dev
| > vite
|
| VITE v8.0.16 ready in 202 ms
|
| → Local:   http://localhost:5173/
| → Network: use --host to expose
| → press h + enter to show help
```

Ingresa al proyecto e instala las dependencias:

```
cd laboratorio-posts-react
npm install
```

Ejecuta el servidor de desarrollo:

```
npm run dev
```

```
P3E004-K@39682082D20P3 MINGW64 ~/AplicacionesWeb/REACT-INTEGRATION
• $ ls
laboratorio-posts-react/

P3E004-K@39682082D20P3 MINGW64 ~/AplicacionesWeb/REACT-INTEGRATION
• $ cd laboratorio-posts-react/

P3E004-K@39682082D20P3 MINGW64 ~/AplicacionesWeb/REACT-INTEGRATION/laboratorio-posts-react
• $ npm install

up to date, audited 136 packages in 1s

31 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

P3E004-K@39682082D20P3 MINGW64 ~/AplicacionesWeb/REACT-INTEGRATION/laboratorio-posts-react
• $ npm run dev

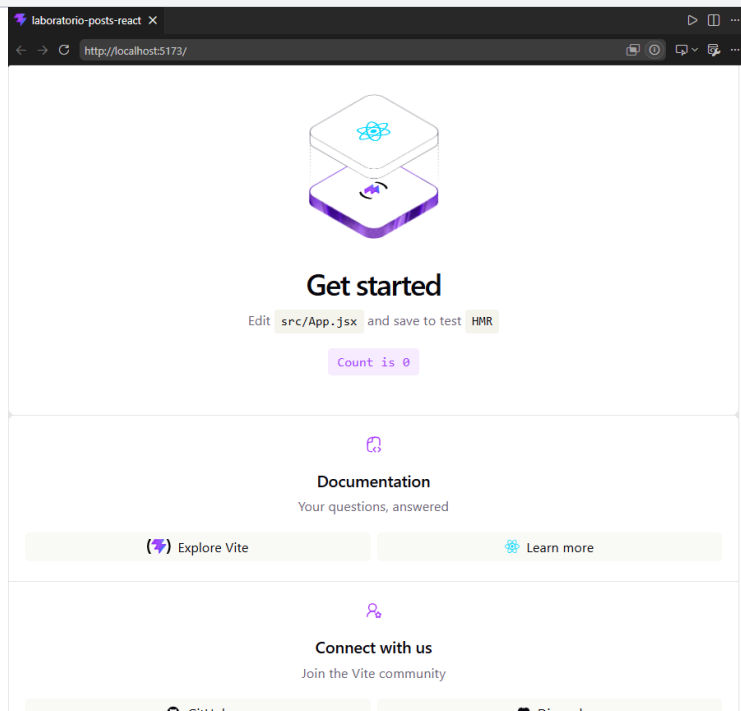
> laboratorio-posts-react@0.0.0 dev
> vite

VITE v8.0.16 ready in 118 ms

→ Local:   http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help
```

Abra en el navegador la URL indicada por Vite, normalmente:

<http://localhost:5173>



## 8. Paso 2: Limpiar el proyecto inicial

Abra el proyecto en Visual Studio Code. Puede usar:

```
code .
```

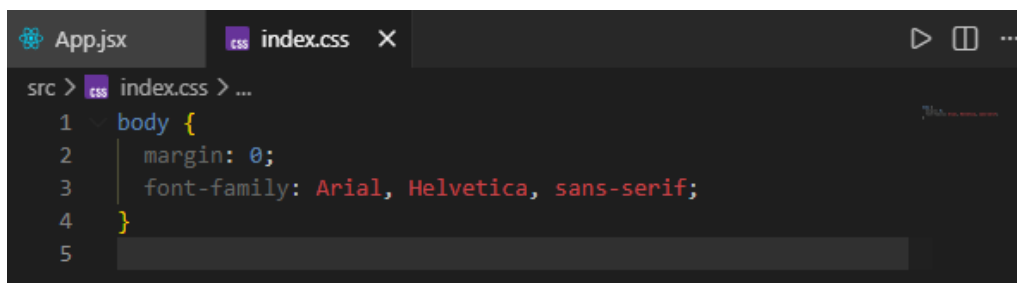
Edite el archivo src/App.jsx y reemplace su contenido por:

```
function App() {  
  return (  
    <main>  
      <h1>Gestor de publicaciones</h1>  
    </main>  
  );  
}  
  
export default App;
```



Luego edite src/index.css, borre el contenido inicial y deje temporalmente:

```
body {  
  margin: 0;  
  font-family: Arial, Helvetica, sans-serif;  
}
```



## 9. Paso 3: Instalar Axios

Dentro del proyecto, ejecute:

```
npm install axios
```

```
P3E004-K@39682082D20P3 MINGW64 C:/Users/P3E004-K/AplicacionesWeb/REACT-INTEGRATI
ON/laboratorio-posts-react
$ npm install axios

added 25 packages, and audited 161 packages in 4s

37 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

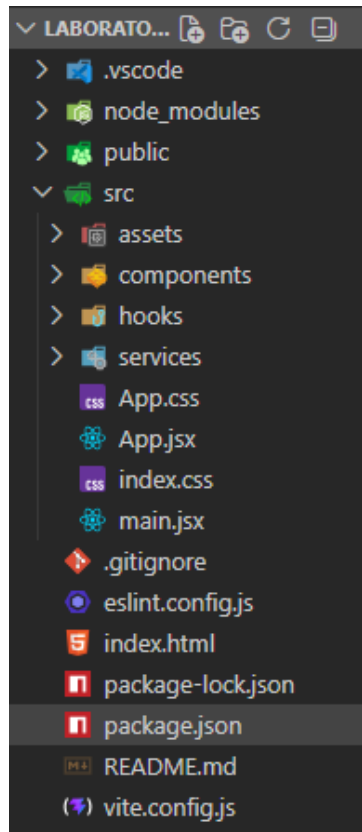
P3E004-K@39682082D20P3 MINGW64 C:/Users/P3E004-K/AplicacionesWeb/REACT-INTEGRATI
ON/laboratorio-posts-react
$
```

Axios será la librería usada para realizar peticiones HTTP a la API.

## 10. Paso 4: Crear la estructura de carpetas

Dentro de src, cree las siguientes carpetas:

```
src/
├── components/
├── hooks/
├── services/
├── App.jsx
├── index.css
└── main.jsx
```



| Carpeta    | Uso                                           |
|------------|-----------------------------------------------|
| components | Componentes visuales de la aplicación         |
| hooks      | Hooks personalizados para lógica reutilizable |
| services   | Comunicación con APIs y configuración HTTP    |

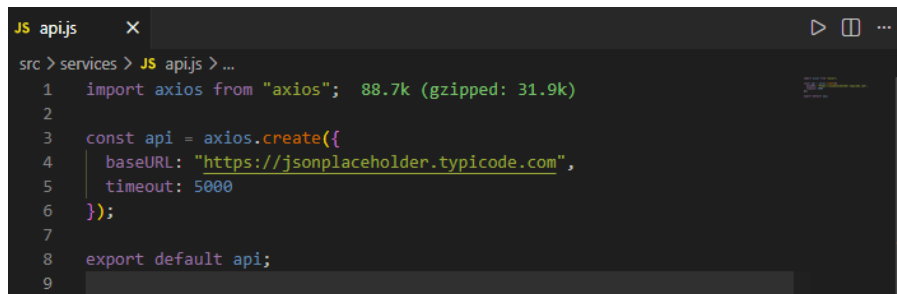
## 11. Paso 5: Crear el servicio base de Axios

Cree el archivo `src/services/api.js` con el siguiente contenido:

```
import axios from "axios";

const api = axios.create({
  baseURL: "https://jsonplaceholder.typicode.com",
  timeout: 5000
});

export default api;
```



Este archivo configura una instancia base de Axios. Gracias a esto, en lugar de escribir la URL completa en cada petición, se podrá usar `api.get("/posts")`.

## 12. Paso 6: Crear el servicio de publicaciones

Cree el archivo `src/services/postService.js` con el siguiente código:

```
import api from "../api";

export async function obtenerPosts() {
  const respuesta = await api.get("/posts");
  return respuesta.data;
}

export async function crearPost(post) {
  const respuesta = await api.post("/posts", post);
  return respuesta.data;
}

export async function eliminarPostApi(id) {
  const respuesta = await api.delete(`/posts/${id}`);
  return respuesta.data;
}
```



}

```

JS postService.js X
src > services > JS postService.js > ...
1  import api from "../api";
2
3  export async function obtenerPosts() {
4    const respuesta = await api.get("/posts");
5    return respuesta.data;
6  }
7
8  export async function crearPost(post) {
9    const respuesta = await api.post("/posts", post);
10   return respuesta.data;
11 }
12
13 export async function eliminarPostApi(id) {
14   const respuesta = await api.delete(`/posts/${id}`);
15   return respuesta.data;
16 }
17

```

Este archivo centraliza las operaciones relacionadas con publicaciones: obtener, crear y eliminar.

### 13. Paso 7: Crear el custom hook usePosts

Cree el archivo src/hooks/usePosts.js. Este hook contendrá la lógica principal del laboratorio.

```

import { useEffect, useState } from "react";
import { obtenerPosts, crearPost, eliminarPostApi } from "../services/postService";

function usePosts() {
  const [posts, setPosts] = useState([]);
  const [cargando, setCargando] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    async function cargarPosts() {
      try {
        setCargando(true);
        setError(null);

        const datos = await obtenerPosts();
        setPosts(datos.slice(0, 10));
      } catch (error) {
        setError("No se pudieron cargar las publicaciones.");
      } finally {
        setCargando(false);
      }
    }

    cargarPosts();
  }, []);

  async function agregarPost(titulo, contenido) {
    const nuevoPost = {
      title: titulo,
      body: contenido,

```

```

        userId: 1
    };

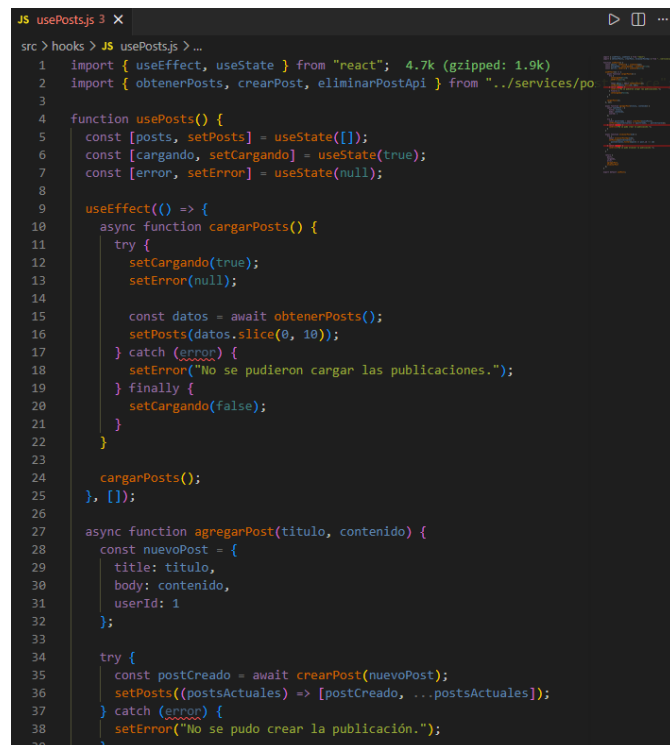
    try {
        const postCreado = await crearPost(nuevoPost);
        setPosts((postsActuales) => [postCreado, ...postsActuales]);
    } catch (error) {
        setError("No se pudo crear la publicación.");
    }
}

async function eliminarPost(id) {
    try {
        await eliminarPostApi(id);
        setPosts((postsActuales) =>
            postsActuales.filter((post) => post.id !== id)
        );
    } catch (error) {
        setError("No se pudo eliminar la publicación.");
    }
}

return {
    posts,
    cargando,
    error,
    agregarPost,
    eliminarPost
};
}

export default usePosts;

```



```

JS usePosts.js 3 X
src > hooks > JS usePosts.js > ...
1 import { useEffect, useState } from "react"; 4.7k (gzipped: 1.9k)
2 import { obtenerPosts, crearPost, eliminarPostApi } from "../services/po
3
4 function usePosts() {
5   const [posts, setPosts] = useState([]);
6   const [cargando, setCargando] = useState(true);
7   const [error, setError] = useState(null);
8
9   useEffect(() => {
10     async function cargarPosts() {
11       try {
12         setCargando(true);
13         setError(null);
14
15         const datos = await obtenerPosts();
16         setPosts(datos.slice(0, 10));
17       } catch (error) {
18         setError("No se pudieron cargar las publicaciones.");
19       } finally {
20         setCargando(false);
21       }
22     }
23
24     cargarPosts();
25   }, []);
26
27   async function agregarPost(titulo, contenido) {
28     const nuevoPost = {
29       title: titulo,
30       body: contenido,
31       userId: 1
32     };
33
34     try {
35       const postCreado = await crearPost(nuevoPost);
36       setPosts((postsActuales) => [postCreado, ...postsActuales]);
37     } catch (error) {
38       setError("No se pudo crear la publicación.");
39     }
40

```

## 14. Explicación del custom hook

### 14.1 Estados principales

```
const [posts, setPosts] = useState([]);
const [cargando, setCargando] = useState(true);
const [error, setError] = useState(null);
```

- posts almacena la lista de publicaciones.
- cargando indica si la aplicación se encuentra solicitando datos.
- error almacena un mensaje si una operación falla.

### 14.2 Carga inicial con useEffect

```
useEffect(() => {
  async function cargarPosts() {
    const datos = await obtenerPosts();
    setPosts(datos.slice(0, 10));
  }

  cargarPosts();
}, []);
```

El arreglo vacío [] indica que el efecto se ejecuta una sola vez cuando se monta el componente que utiliza este hook.

### 14.3 Agregar y eliminar publicaciones

Para agregar se usa el operador spread, colocando la nueva publicación al inicio de la lista. Para eliminar se usa filter, creando una nueva lista sin el elemento seleccionado.

```
setPosts((postsActuales) => [postCreado, ...postsActuales]);

setPosts((postsActuales) =>
  postsActuales.filter((post) => post.id !== id)
);
```

## 15. Paso 8: Crear el componente GestorPosts

Cree el archivo src/components/GestorPosts.jsx con el siguiente contenido:

```
import { useState } from "react";
import usePosts from "../hooks/usePosts";

function GestorPosts() {
  const {
    posts,
    cargando,
    error,
    agregarPost,
    eliminarPost
  } = usePosts();
```

```

const [titulo, setTitulo] = useState("");
const [contenido, setContenido] = useState("");

async function manejarEnvio(evento) {
  evento.preventDefault();

  if (titulo.trim() === "" || contenido.trim() === "") {
    alert("Debe completar el título y el contenido.");
    return;
  }

  await agregarPost(titulo, contenido);

  setTitulo("");
  setContenido("");
}

if (cargando) {
  return (
    <section className="card">
      <p>Cargando publicaciones...</p>
    </section>
  );
}

return (
  <section className="card">
    <h2>Gestor de publicaciones</h2>

    {error && <p className="error">{error}</p>}

    <form onSubmit={manejarEnvio} className="formulario">
      <label htmlFor="titulo">Título:</label>
      <input
        id="titulo"
        type="text"
        value={titulo}
        onChange={(e) => setTitulo(e.target.value)}
        placeholder="Ingrese el título"
      />

      <label htmlFor="contenido">Contenido:</label>
      <textarea
        id="contenido"
        value={contenido}
        onChange={(e) => setContenido(e.target.value)}
        placeholder="Ingrese el contenido"
        rows="4"
      />

      <button type="submit">Crear publicación</button>
    </form>

    <hr />

    <h3>Publicaciones registradas</h3>
  )
);

```

```

    {posts.length === 0 ? (
      <p>No existen publicaciones.</p>
    ) : (
      <div className="lista-posts">
        {posts.map((post) => (
          <article className="post" key={post.id}>
            <h4>{post.title}</h4>
            <p>{post.body}</p>

            <button
              className="btn-eliminar"
              onClick={() => eliminarPost(post.id)}
            >
              Eliminar
            </button>
          </article>
        ))}
      </div>
    )}
  </section>
);
}

export default GestorPosts;

```

```

GestorPosts.jsx X
src > components > GestorPosts.jsx > ...
1  import { useState } from "react"; 4.6k (gzipped: 1.9k)
2  import usePosts from "../hooks/usePosts";
3
4  function GestorPosts() {
5    const {
6      posts,
7      cargando,
8      error,
9      agregarPost,
10     eliminarPost
11   } = usePosts();
12
13   const [titulo, setTitulo] = useState("");
14   const [contenido, setContenido] = useState("");
15
16   async function manejarEnvio(evento) {
17     evento.preventDefault();
18
19     if (titulo.trim() === "" || contenido.trim() === "") {
20       alert("Debe completar el título y el contenido.");
21       return;
22     }
23

```

## 16. Explicación del componente GestorPosts

- Usa el custom hook usePosts para recibir posts, cargando, error, agregarPost y eliminarPost.
- Usa useState para controlar los campos título y contenido del formulario.
- Usa preventDefault para evitar que el formulario recargue la página.
- Valida que los campos no estén vacíos.
- Muestra un mensaje de carga si todavía se están consultando datos.
- Muestra un mensaje de error si ocurre un problema.
- Renderiza la lista de publicaciones con map.
- Elimina una publicación llamando a eliminarPost con el id correspondiente.

## 17. Paso 9: Usar el componente en App.jsx

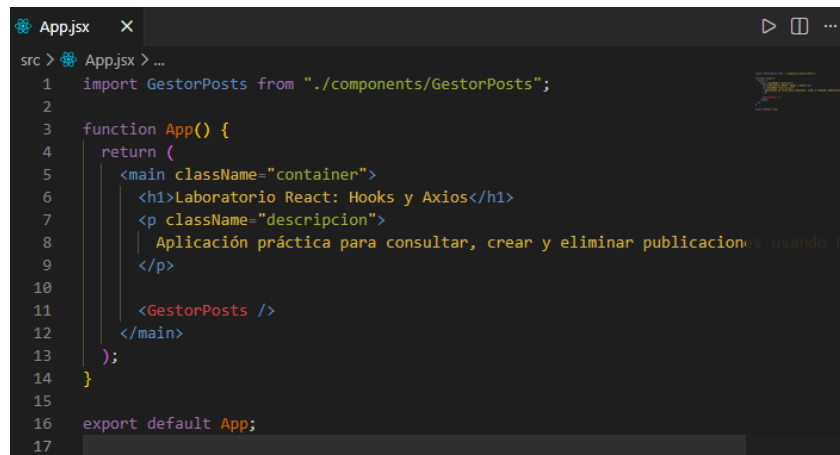
Abra src/App.jsx y reemplace su contenido por:

```
import GestorPosts from "../components/GestorPosts";

function App() {
  return (
    <main className="container">
      <h1>Laboratorio React: Hooks y Axios</h1>
      <p className="descripcion">
        Aplicación práctica para consultar, crear y eliminar publicaciones usando React, Hooks y Axios.
      </p>

      <GestorPosts />
    </main>
  );
}

export default App;
```



## 18. Paso 10: Agregar estilos CSS

Abra src/index.css y reemplace el contenido por:

```
* {
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: Arial, Helvetica, sans-serif;
  background: #f3f4f6;
  color: #111827;
}

.container {
  max-width: 1000px;
  margin: 40px auto;
  padding: 20px;
```

```
}

.descripcion {
  color: #4b5563;
  margin-bottom: 24px;
}

.card {
  background: white;
  padding: 24px;
  border-radius: 14px;
  box-shadow: 0 2px 10px rgba(0, 0, 0, 0.1);
}

.formulario {
  margin-top: 20px;
}

label {
  font-weight: bold;
  display: block;
  margin-bottom: 6px;
}

input,
textarea {
  width: 100%;
  padding: 10px;
  margin-bottom: 16px;
  border: 1px solid #d1d5db;
  border-radius: 8px;
  font-family: inherit;
}

button {
  background: #2563eb;
  color: white;
  border: none;
  padding: 10px 16px;
  border-radius: 8px;
  cursor: pointer;
  font-weight: bold;
}

button:hover {
  background: #1d4ed8;
}

.btn-eliminar {
  background: #dc2626;
}

.btn-eliminar:hover {
  background: #b91c1c;
}

.error {
```

```

background: #fee2e2;
color: #991b1b;
padding: 12px;
border-radius: 8px;
}

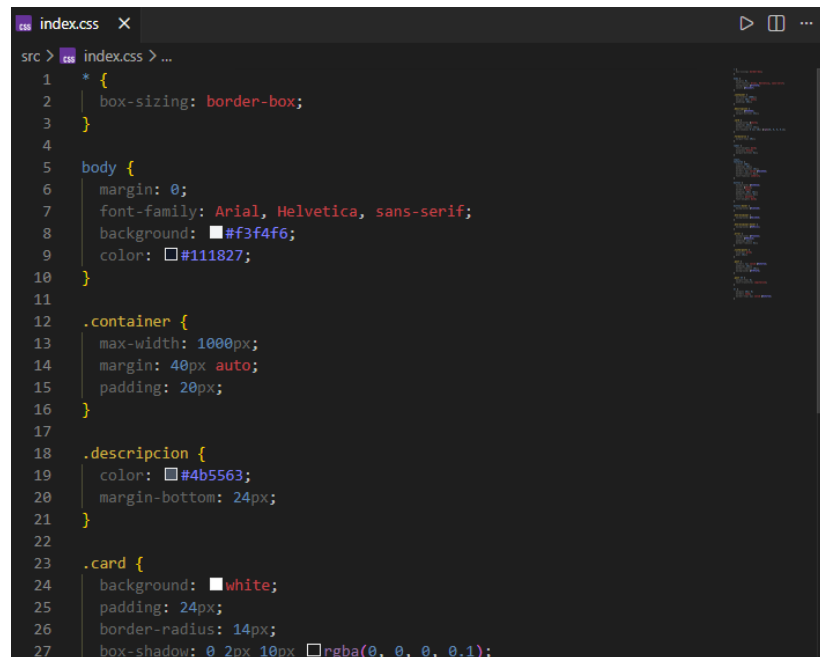
.lista-posts {
  display: grid;
  gap: 16px;
}

.post {
  border: 1px solid #e5e7eb;
  padding: 16px;
  border-radius: 10px;
  background: #f9fafb;
}

.post h4 {
  margin-top: 0;
  text-transform: capitalize;
}

hr {
  margin: 28px 0;
  border: none;
  border-top: 1px solid #e5e7eb;
}

```



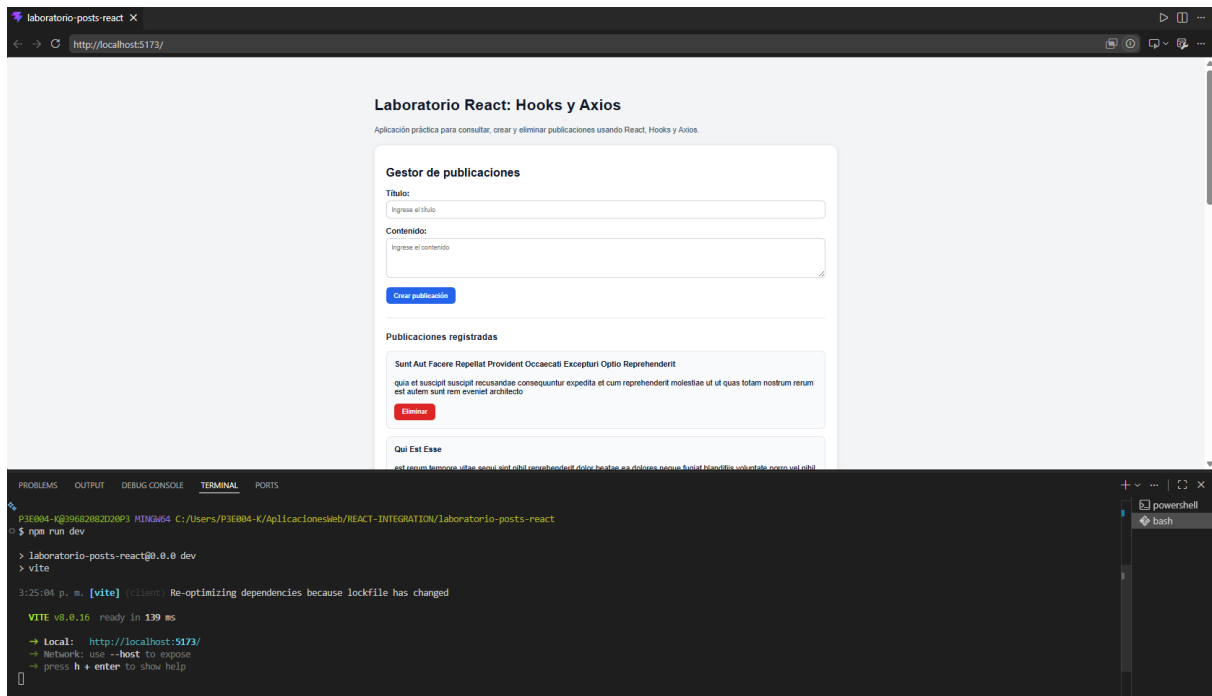
## 19. Paso 11: Ejecutar el proyecto

En la terminal, ejecute:

```
npm run dev
```



Abra la URL mostrada por Vite. Debe observar la aplicación con el formulario y la lista de publicaciones.

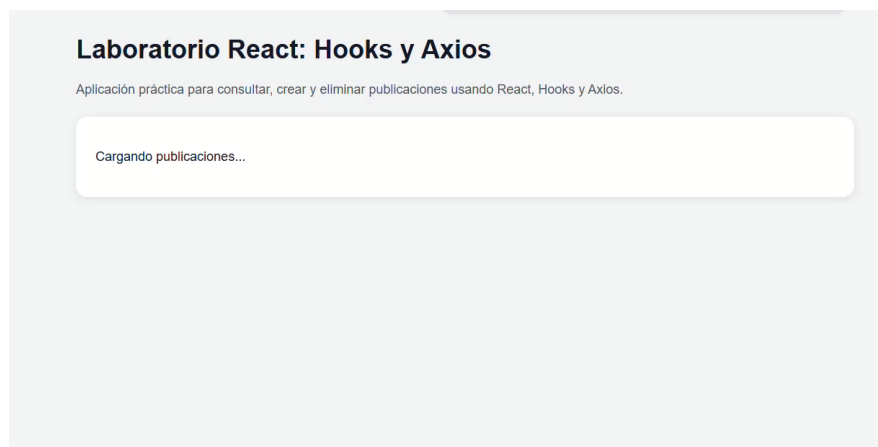


## 20. Pruebas del laboratorio

### 20.1 Prueba de carga inicial

Al abrir la aplicación debe aparecer brevemente el mensaje:

Cargando publicaciones...



Luego deben mostrarse las publicaciones obtenidas desde la API.

### 20.2 Prueba de creación

Ingrese los siguientes datos:

Título: Mi primera publicación

Contenido: Esta publicación fue creada desde React.

## Laboratorio React: Hooks y Axios

Aplicación práctica para consultar, crear y eliminar publicaciones usando React, Hooks y Axios.

### Gestor de publicaciones

Título:

Mi primera publicación

Contenido:

Esta publicación fue creada desde React.

Crear publicación

### Gestor de publicaciones

Título:

Ingrese el título

Contenido:

Ingrese el contenido

Crear publicación

#### Publicaciones registradas

Mi Primera Publicación

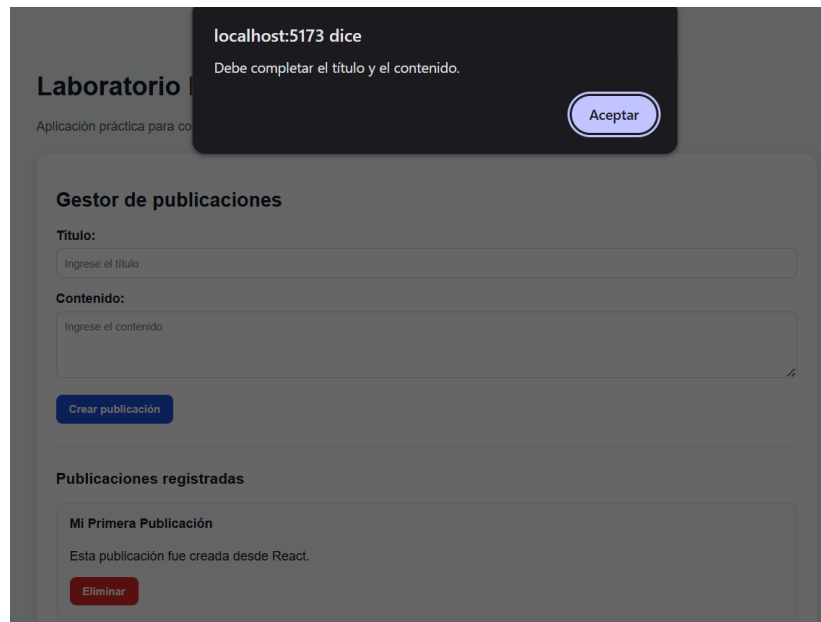
Esta publicación fue creada desde React.

Eliminar

Al presionar Crear publicación, la publicación debe aparecer al inicio de la lista y el formulario debe limpiarse.

### 20.3 Prueba de validación

Deje los campos vacíos y presione Crear publicación. Debe aparecer el mensaje de alerta correspondiente.



## 20.4 Prueba de eliminación

Presione Eliminar en una publicación. La publicación debe desaparecer de la lista local.

### Publicaciones registradas

#### Mi Primera Publicación

Esta publicación fue creada desde React.

Eliminar

#### Sunt Aut Facere Repellat Provident Occaecati Excepturi Optio Reprehenderit

quia et suscipit suscipit recusandae consequuntur expedita et cum reprehenderit molestiae ut ut quas totam nostrum rerum est autem sunt rem eveniet architecto

Eliminar

### Publicaciones registradas

#### Sunt Aut Facere Repellat Provident Occaecati Excepturi Optio Reprehenderit

quia et suscipit suscipit recusandae consequuntur expedita et cum reprehenderit molestiae ut ut quas totam nostrum rerum est autem sunt rem eveniet architecto

Eliminar

#### Qui Est Esse

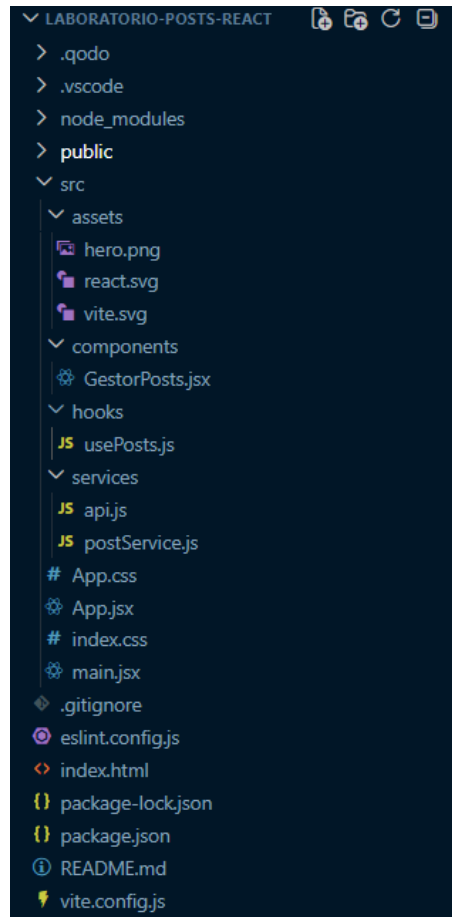
est rerum tempore vitae sequi sint nihil reprehenderit dolor beatae ea dolores neque fugiat blanditiis voluptate porro vel nihil

## 21. Estructura final del proyecto

```
src/
|
├── components/
```

```

├── GestorPosts.jsx
├── hooks/
│   └── usePosts.js
├── services/
│   ├── api.js
│   └── postService.js
├── App.jsx
├── index.css
└── main.jsx
    
```



## 22. Flujo de funcionamiento

```

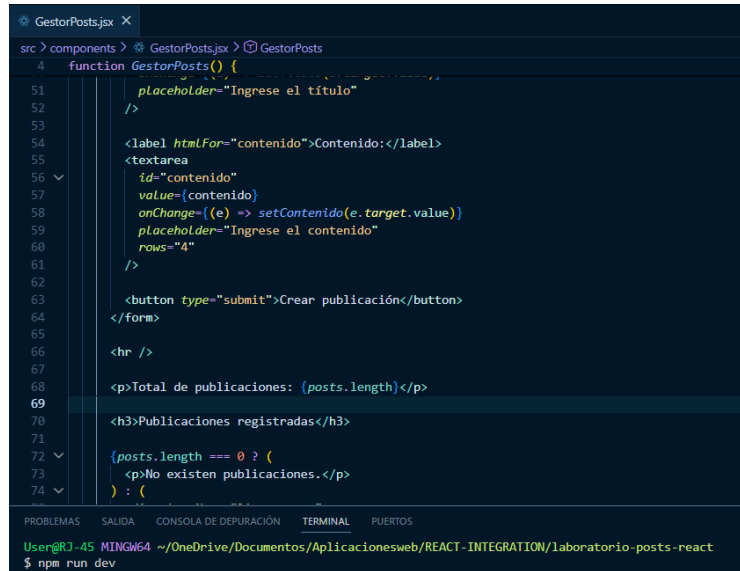
App.jsx
  ↓
GestorPosts.jsx
  ↓
usePosts.js
  ↓
postService.js
  ↓
api.js
  ↓
JSONPlaceholder API
    
```

App.jsx muestra la aplicación principal. GestorPosts.jsx contiene la interfaz. usePosts.js contiene la lógica. postService.js contiene las funciones HTTP específicas. api.js contiene la configuración base de Axios.

## 23. Actividades de mejora

### Actividad 1: Mostrar cantidad de publicaciones

```
<p>Total de publicaciones: {posts.length}</p>
```



```

4  function GestorPosts() {
51     placeholder="Ingrese el título"
52     />
53
54     <label htmlFor="contenido">Contenido:</label>
55     <textarea
56       id="contenido"
57       value={contenido}
58       onChange={(e) => setContenido(e.target.value)}
59       placeholder="Ingrese el contenido"
60       rows="4"
61     />
62
63     <button type="submit">Crear publicación</button>
64   </form>
65
66   <hr />
67
68   <p>Total de publicaciones: {posts.length}</p>
69
70   <h3>Publicaciones registradas</h3>
71
72   {posts.length === 0 ? (
73     <p>No existen publicaciones.</p>
74   ) : (

```

### Actividad 2: Agregar un campo autor

Modifique el formulario para incluir el campo Autor y guarde el autor junto con cada publicación.



```

function GestorPosts() {
  const {
    posts,
    cargando,
    error,
    agregarPost,
    eliminarPost
  } = usePosts();

  const [titulo, setTitulo] = useState("");
  const [autor, setAutor] = useState("");
  const [contenido, setContenido] = useState("");

  async function manejarEnvio(evento) {
    evento.preventDefault();

    if (titulo.trim() === "" || autor.trim() === "" || contenido.trim() === "") {
      alert("Debe completar el título, el autor y el contenido.");
      return;
    }

```

### Gestor de publicaciones

**Título:**

**Autor:**

**Contenido:**

Crear publicación

---

Total de publicaciones: 10

**Publicaciones registradas**

**Sunt Aut Facere Repellat Provident Occaecati Excepturi Optio Reprehenderit**

**Autor:** Sin autor

quia et suscipit suscipit recusandae consequuntur expedita et cum reprehenderit molestiae ut ut quas totam nostrum rerum est autem sunt rem eveniet architecto

Eliminar

### Actividad 3: Confirmar antes de eliminar

```
const confirmar = confirm("¿Está seguro de eliminar esta publicación?");
```

**Contenido:**

Crear publicación

localhost:5173 dice

¿Está seguro de eliminar esta publicación?

Aceptar Cancelar

---

Total de publicaciones: 10

**Publicaciones registradas**

**Sunt Aut Facere Repellat Provident Occaecati Excepturi Optio Reprehenderit**

**Autor:** Sin autor

quia et suscipit suscipit recusandae consequuntur expedita et cum reprehenderit molestiae ut ut quas totam nostrum rerum est autem sunt rem eveniet architecto

Eliminar

**Qui Est Esse**

### Actividad 4: Limitar caracteres

- El título debe tener mínimo 5 caracteres.
- El contenido debe tener mínimo 10 caracteres.

```
const autorLimpio = autor.trim();
const contenidoLimpio = contenido.trim();

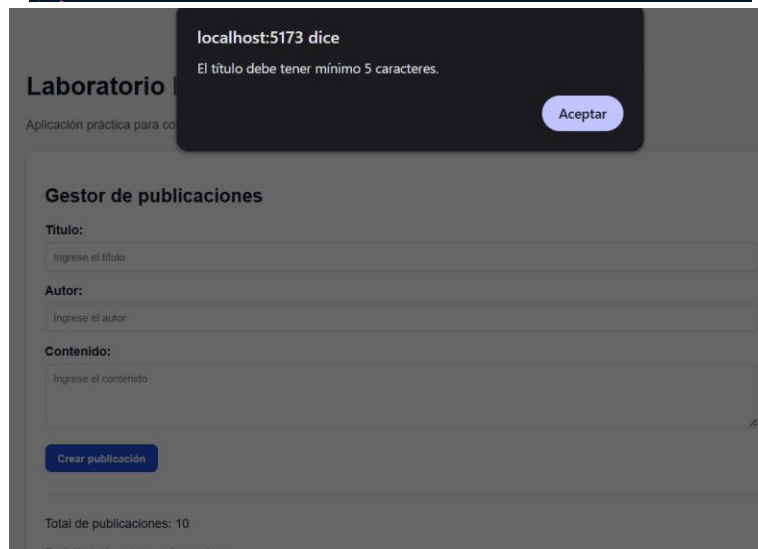
if (tituloLimpio.length < 5) {
  alert("El título debe tener mínimo 5 caracteres.");
  return;
}

if (autorLimpio === "") {
  alert("Debe completar el autor.");
  return;
}

if (contenidoLimpio.length < 10) {
  alert("El contenido debe tener mínimo 10 caracteres.");
  return;
}

await agregarPost(tituloLimpio, autorLimpio, contenidoLimpio);

setTitulo("");
setAutor("");
setContentido("");
```



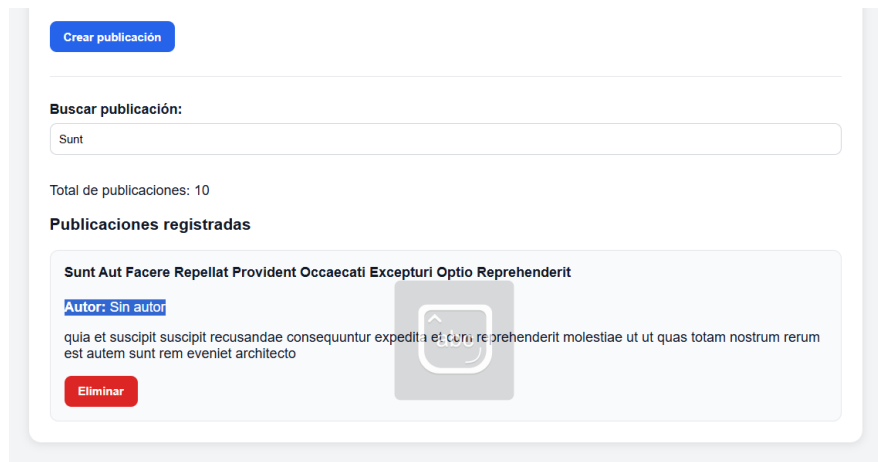
## Actividad 5: Buscador de publicaciones

```
const postsFiltrados = posts.filter((post) =>
  post.title.toLowerCase().includes(búsqueda.toLowerCase())
);
```

```
<button type="submit">Crear publicación</button>
</form>

<label htmlFor="busqueda">Buscar publicación:</label>
<input
  id="busqueda"
  type="text"
  value={busqueda}
  onChange={(e) => setBusqueda(e.target.value)}
  placeholder="Buscar por título"
/>

<hr />
```



## 24. Preguntas de análisis

### 1. ¿Para qué sirve useState en este laboratorio?

useState sirve para guardar datos que cambian mientras usamos la página, como el título, el autor, el contenido o la búsqueda. Así React puede actualizar lo que ve el usuario cuando esos valores cambian.

### 2. ¿Por qué usamos useEffect para cargar las publicaciones?

useEffect se usa para cargar las publicaciones cuando el componente aparece en pantalla. Eso evita tener que hacerlo manualmente cada vez y hace que la carga ocurra en el momento correcto.

### 3. ¿Qué diferencia hay entre api.js y postService.js?

api.js crea y configura la conexión con la API, como la dirección base y el tiempo de espera. postService.js usa esa conexión para hacer acciones concretas, como traer, crear o eliminar publicaciones.

### 4. ¿Por qué separamos la lógica en un custom hook?

Separamos la lógica en un custom hook para que el componente quede más ordenado y fácil de leer. Así, la parte de manejar datos y la parte visual no quedan mezcladas.

### 5. ¿Qué hace setPosts(datos.slice(0, 10))?

setPosts(datos.slice(0, 10)) guarda solo las primeras 10 publicaciones que llegaron de la API. Es una forma de mostrar una cantidad limitada.

### 6. ¿Qué hace filter cuando eliminamos una publicación?

filter crea una nueva lista sin la publicación que se eliminó. Básicamente, conserva las publicaciones cuyo id no coincide con el que se borró.

### 7. ¿Qué ocurre si la API no responde?



Si la API no responde, se muestra un mensaje de error y la app evita romperse. En este laboratorio, el usuario ve que no se pudieron cargar o crear las publicaciones.

8. ¿Por qué se usa `preventDefault()` en el formulario?

Axios facilita hacer peticiones porque ya trae varias cosas listas, como manejar respuestas JSON, errores y configuración general. Con `fetch` se puede hacer lo mismo, pero normalmente hay que escribir más código.

9. ¿Qué ventaja tiene Axios frente a escribir todo directamente con `fetch`?

Axios facilita hacer peticiones porque ya trae varias cosas listas, como manejar respuestas JSON, errores y configuración general. Con `fetch` se puede hacer lo mismo, pero normalmente hay que escribir más código.

10. ¿Qué parte del proyecto se encarga de la interfaz visual?

La parte que se encarga de la interfaz visual es el componente de React, especialmente `GestorPosts`. Ahí se ven el formulario, la lista de publicaciones y los botones.

## 25. Rúbrica de evaluación sugerida

| Criterio                                                                               | Puntaje |
|----------------------------------------------------------------------------------------|---------|
| Crea correctamente el proyecto con Vite                                                | 1       |
| Instala y configura Axios                                                              | 1       |
| Organiza carpetas <code>components</code> , <code>hooks</code> y <code>services</code> | 1       |
| Consume publicaciones desde la API                                                     | 2       |
| Maneja estados de carga y error                                                        | 2       |
| Implementa formulario controlado                                                       | 2       |
| Crea publicaciones simuladas                                                           | 2       |
| Elimina publicaciones de la lista                                                      | 2       |
| Usa correctamente un custom hook                                                       | 2       |
| Aplica estilos CSS básicos                                                             | 1       |
| Explica el funcionamiento del código                                                   | 2       |
| Total                                                                                  | 18      |

## 26. Conclusión

En este laboratorio se construyó una aplicación React completa y organizada usando React, Hooks, Axios, servicios y custom hooks. La combinación useState + useEffect + Axios permite construir aplicaciones que consultan APIs, muestran datos dinámicos, trabajan con formularios y actualizan la interfaz de forma controlada.

## 27. Link Repositorio

<https://github.com/7Lemaitre7/react-implementacion.git>